

Keynote · 2026

Things you don't know about Agents

Loops, harness, context, memory: what
actually moves the needle in production.

kami slides demo · A4 landscape · 8 slides

Agent Engineering

Simple core, complex surroundings

The loop is small. The **infrastructure** around it is what keeps it stable as features grow.

0.

A working Agent loop fits in about 20 lines of code.

1.

LOOP

Control flow lives in the tools, not in branchy internal state.

2.

Workflow vs Agent: predefined paths in code vs model picks paths at runtime.

If your loop keeps growing every sprint, you are fixing the wrong layer. The tax is paid **outside** the loop.

One turn, traced

The agent loop, made concrete: a single user turn flows through a tool and back.



Harness wins over hardware

More expensive models bring gains **far smaller than expected**. Verification, boundaries, feedback, and fallbacks matter more than model capability.

Tool

0.

Upgrade the harness first. If accuracy does not move, then try the model.

1.

Evaluation is the only honest signal. Test harness before test model.

2.

result in

Smaller model + strong harness routinely beats larger model + weak harness production.

20

feedback, lines of code in a working Agent core loop

Agent Engineering

4

harness layers that matter: verify, bound, fallback

03

Density beats length

Long context windows do not fix weak context design. Context Rot sets in around

85%

0.

Layer the load: constant, on-demand, runtime, memory, system.

y 53%
C
a
r
u
c
A

1.

Index first, full content on demand. Beats dumping everything up front.

2.

Stable prompt prefixes let caching actually pay off.

3.

Every token that is not load-bearing is diluting signal.

10 ×

velocity gains trace to execution discipline, not model swaps

Put state outside the context

Tools should match Agent goals. Memory belongs on disk, outside the context window.

0. ACI principle : design for the Agent's goal, not the HTTP verb.
1. Bad tool descriptions masquerade as model failures.
2. File-based state survives restarts. In-context state does not.

consistency needs explicit consolidation,

Four kinds

0. Working
1. Procedural
2. Episodic
3. Semantic sessions,

Cross-session not hope.

Pseudocode over syntax

Comments should outnumber code lines. The reader sees **logic**, not a language tutorial.

```
tool call or decide to stop
picks the next action
= 0.
```

Write the intent first, then the implementation detail below
: the context, window - fast, expensive, temporary.

```
1.
: SKILL.md files - how to behave, loaded lazily.
  Variable names describe what a thing is, not what type it
  is.
= tools[action.name](action.args)
: JSONL logs - what happened, appendable.
```

One concept per block. Split ruthlessly.
: MEMORY.md - what to remember across

```
# resolve
function
  # model
  action

# terminal
if
  return

# delegate
result
return
```

```
agent_step(context, tools):
```

```
    model_think(context)
```

```
    if action_type == "stop":
```

```
        return action_result
```

```
    agent_step(context + result, tools)
```

the Agent. Most
noise.

Protocol first Then parallelis

Fix your evals before you tweak
apparent model failures are infrastructure

